面向 PTCG 的机制感知智能体
双墙协同、局面响应与分层决策设计

Mechanism-Aware PTCG Agent: Dual-Wall Coordination, Context-Aware Response, and Hierarchical Decision Design

作者：山里灵活的狗（队伍 **47**）
文稿类型：智能体系统设计论文
日期：**2026** 年 **10** 月 **24** 日

摘要

针对宝可梦集换式卡牌环境中防御机制异质、行动顺序耦合以及资源用途随局面变化的问题，本文设计一种机制感知的轻量规则智能体。系统将攻击伤害与招式效果作为不同的防御对象，以 Crustle 和 Team Rocket's Articuno 构成互补防御核心，并利用 Mist Energy 为关键持有者补充效果保护；在此基础上，以公开场面信号选择防御或低能耗反击线路。决策层通过攻击与普通准备动作的评分梯度、关键分支保留和己方备战误伤扣分，组织当前回合的动作优先级。资源层将进化前置供能、攻击就绪状态、关键组件保留与资源回收相连接；执行层按选择类型分派处理，并通过索引范围、去重与数量约束规范化输出。本文的主要设计贡献是：将防御语义转化为卡组角色分工，将可见威胁转化为资源与动作优先级，并将策略判断与执行约束组成可追踪的决策流程。文中重点说明已实现的设计、参数与模块对应关系。

关键词： 机制感知；双墙协同；局面响应；动作优先级；资源保护；规则智能体

1 引言

卡牌智能体需要同时处理规则解释、资源积累和动作选择。防御卡牌并非共享同一种“免疫”语义：阻止招式伤害的能力，不一定能够阻止放置伤害指示物的效果；效果保护也不意味着可以承受任意直接攻击。与此同时，铺场、进化和供能通常存在依赖关系，出战位与备战位又会竞争有限资源。如果这些因素分别由互不关联的规则处理，策略可能拥有所需组件，却无法将它们组织成当前可执行的计划。

本项目围绕“识别攻击路径、匹配保护条件、建立可执行攻击能力”组织智能体。设计从卡组层延伸到决策与执行层：卡组提供互补角色，观察模块识别公开信号，动作评分明确当前优先级，资源规则维持关键组件，输出规范化则约束最终接口行为。各模块共享当前局面，而不是让同一张牌在所有情况下都具有固定价值。

本文将本项目的设计贡献概括为五项：第一，基于伤害与效果语义的双墙互补；第二，基于公开信号的威胁预判与策略响应；第三，攻击与准备动作的分层评分及关键分支保留；第四，进化供能与关键资源保护相结合的资源组织；第五，类型化选择分派与合法输出约束的执行体系。这里的创新点指本项目实现中的设计与集成贡献，各项内容均与具体代码模块对应。

2 总体架构与决策对象

2.1 从卡组角色到动作输出

系统由角色层、局面层、决策层、资源层和执行层组成。角色层定义伤害防御、效果防御与后备反击的分工；局面层从己方手牌和双方公开信息提取上下文；决策层对合法候选进行评分；资源层参与检索、弃牌、供能和回收的取舍；执行层按选择类型生成索引并进行规范化。图 1 展示了这些模块的连接方式。

2.2 观察边界与类型化选择

设第 t 次接口调用的观察为 o_t，其中包括自己的手牌、双方公开出战与备战、公开资源计数、当前选择类型及候选列表。对手未公开手牌内容与未来牌序不参与策略计算。这里的接口调用次数不等同于回合数，同一回合可能先后进入主动作、卡牌、能量或攻击等不同选择。

类型化分派将“选择什么操作”和“该操作具体选择哪些对象”分开处理。例如，主动作决定使用检索卡，后续卡牌选择再依据检索上下文确定目标；贴能与撤退也由相应选择对象和费用条件约束。这样可以复用共同的局面表示，同时保留不同操作的语义。

3 创新一：以防御语义组织双墙协同

3.1 将保护范围拆解为条件关系

双墙设计首先区分保护对象、伤害来源和效果类型。Crustle（345）的 Mysterious Rock Inn 防止对方 Pokémon ex 招式对自身造成的伤害，承担特定直接伤害路径的出战防御；Team Rocket's Articuno（414）



图 1　双墙角色分工与共享上下文架构。保护语义、后备攻击和输出流程共同围绕当前局面组织；各保护组件仍按其卡牌条件生效。

的 Repelling Veil 则保护己方基础 Team Rocket's Pokémon 免受对方招式效果，Articuno 自身满足该条件。两者处理的攻击路径不同，因而具有互补性。卡牌机制依据平台卡表核对。平台卡牌资料

Mist Energy（11）进一步把效果保护附着到具体持有者，使 Crustle 等其他线路也可以补充相应保护。这形成了“固定角色能力 + 持有者保护组件”的组合方式：既可依赖 Articuno 自带的角色条件，也可通过能量配置改变另一只宝可梦的保护状态。Hero’s Cape 提供承伤空间，Switch 为角色换位提供操作入口。表 **1**　防御组件及其设计角色。

组件	角色职责	生效条件与边界
Crustle 345	承接适用的 ex 招式伤害	不自动覆盖非 ex 伤害、招式效果或无视防御效果的攻击
Team Rocket's Articuno 414	承接适用的招式效果	保护范围为己方基础 Team Rocket's Pokémon，直接伤害另行处理
Mist Energy 11	为持有者补充效果保护	保护随持有者与附着状态成立，不能代替直接伤害防御
Hero’s Cape 1159	增加关键角色的承伤空间	属于生命值支持组件，不改变伤害与效果的语义
Switch 1123	让适配角色进入出战位	需结合己方备战角色与当前保护需求选择目标

3. 2 将卡组互补转化为动作互补

角色互补需要通过检索、部署和换位才能成为实际决策。系统在需要效果保护且场上尚无对应角色时，提高合适检索与选牌分支的优先级；当适配的保护角色位于备战位时，换位选择考虑其保护潜力；当出战角色已经匹配当前攻击路径时，撤退评分避免轻易破坏这一配置。

这一设计将同一保护需求贯穿多个操作，而非只在铺场阶段处理一次。它的创新落点是卡组构筑与动作控制共享角色语义：寻找哪个组件、让谁出战、给谁配置资源，围绕同一个局面目标协同。

3. 3 防御之外保留低能耗反击

最终卡组保留 2-2 Bunnelby / Diggersby 线路，作为防御匹配不足时的攻击选择。系统在对方可见场面满足非 ex 竞速条件、且不属于已识别效果防御需求时，提高该线路的进化、升出战位和供能优先级。双墙因此承担有条件的防御职责，后备线路承担低能耗反击职责，避免所有资源都围绕单一角色积累。

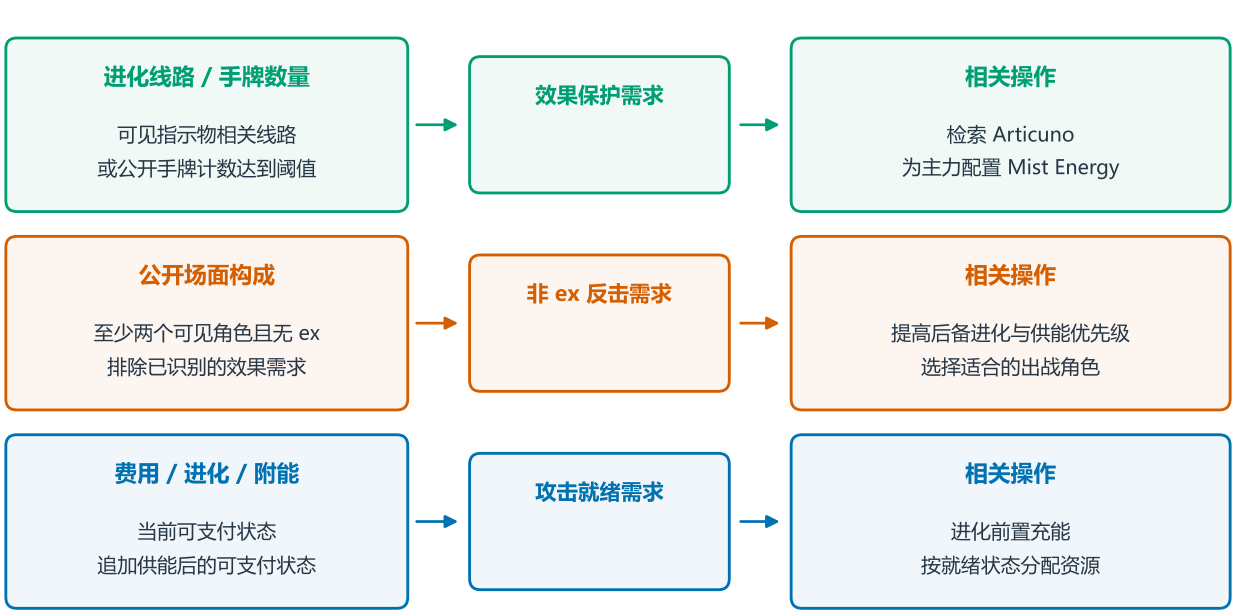


图 2　公开信号到操作关注点的映射。图示为已有上下文分支的组织方式，各分支可共同影响选择，并非独立学习的互斥状态机。

4 创新二：以公开信号驱动威胁预判与策略响应

4.1 将可见信号转换为操作需求

系统使用对方公开进化线路、出战与备战构成、手牌数量和招式文本特征推断当前需要关注的攻击路径。例如，`need_effect_wall` 检查已知指示物相关进化线路，也将对方公开手牌计数达到阈值作为补充信号；`opp_hand_scale` 则检查可见宝可梦的招式是否与手牌数量相关。

这些规则允许在最终攻击角色尚未就位时提前调整检索目标。手牌数量属于辅助信号，并不等同于获知对方手牌内容或确定其卡组；系统仍以当前可见场面和已编码机制作出启发式判断。图 2 将公开信号、策略需求和相应操作连接起来。

4.2 以条件分支表达防御与反击模式

`non_ex_race` 要求对方至少出现两个可见角色、其中没有 ex，并排除已识别的手牌数量相关攻击与效果保护需求。该条件触发时，后备线路获得额外资源优先级。条件不满足时，系统仍分别评估出战伤害、保护组件和当前攻击能力。

这里的“模式”由共享上下文的条件分支表达，不同分支可以共同影响候选排序。它的设计价值在于把可见信号直接映射到不同操作的关注点，使进化、检索、贴能与换位围绕同一需求调整。

4.3 将保护潜力纳入升出战位选择

升出战位的判断不仅依赖攻击输出。代码先用 `ready_value` 估计候选角色的当前攻击能力、已投入能量与有效生命值，再用 `immunity_bonus` 补充当前攻击路径下的保护潜力；满足非 ex 反击条件的后备线路另获加分。

其基础就绪评分可概括为

$$R(p) = 0.8\hat{d}(p) + 45e(p) + 0.25\hat{h}(p) + B(p) \tag{1}$$

其中，$\hat{d}(p)$ 为当前可支付招式的伤害估计，$e(p)$ 为代码计入的附能数量，$\hat{h}(p)$ 为代码估计的有效生命值，$B(p)$ 为无攻击能力时的扣分与满足特定供能数量条件时的加分。保护潜力是在此基础上加入的局面项，并非生命值越高就必然越适合出战。伤害与保护估计仍受已编码规则覆盖范围约束。

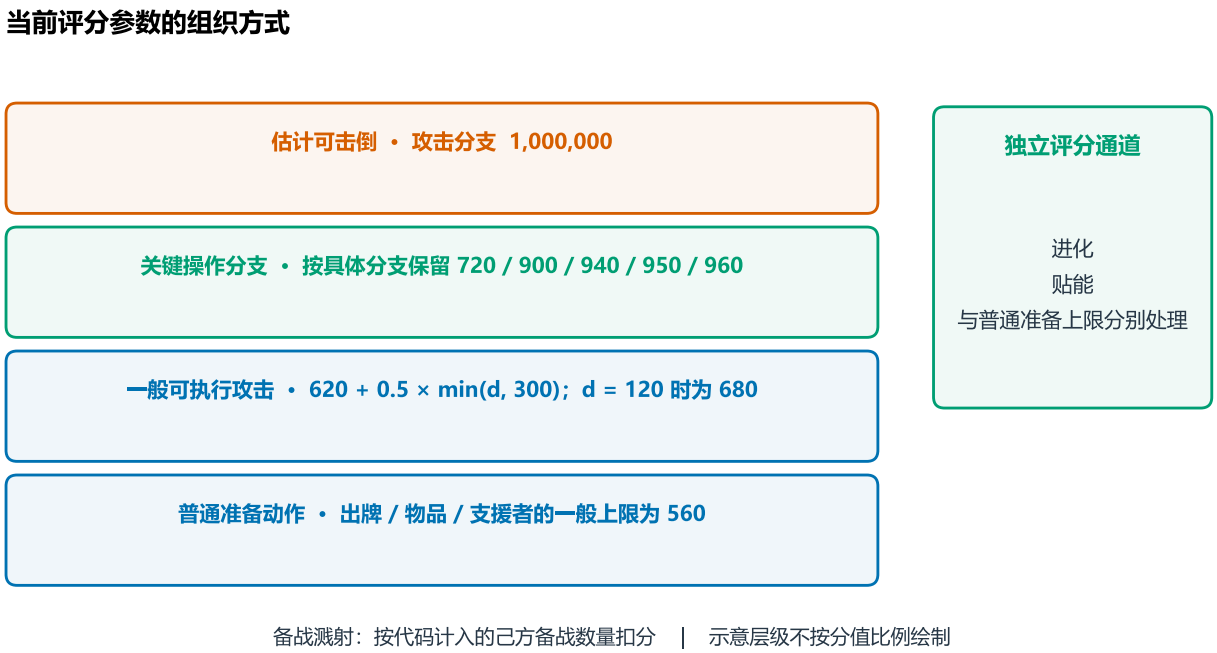


图 3　当前实现的评分梯度。数值是代码参数与公式示例，示意层级不按分值比例绘制；进化和贴能使用独立评分通道。

5 创新三：分层评分组织攻击、准备与关键动作

5.1 用评分带表达当前回合的行动关系

主动作候选在同一评分空间内排序。设计中需要区分当前可兑现的攻击、普通准备动作和必须保留优先级的关键分支，因此将攻击基础评分置于普通准备动作上限之上，同时保留特定高价值分支。图 3 展示当前实现中的典型评分关系；图中数值是代码参数与公式代入，不是对局测量值。

设 $\hat{d}$ 为估计伤害，在攻击条件检查通过后，评分为

$$s_{\text{atk}} = \begin{cases} 10^6 & \text{估计可击倒当前目标} \\ \max\left\{120,\, 620 + 0.5\min\left(\hat{d}, 300\right) - 30b\right\} & \text{匹配己方备战溅射文本} \\ 620 + 0.5\min\left(\hat{d}, 300\right) & \text{其他情况} \end{cases} \tag{2}$$

其中 b 为代码计入的己方备战数量；攻击条件未满足时，相应候选得分为 0。对于估计造成 120 点伤害且不击倒的普通攻击，代入式（2）得到 680 分。备战溅射分支显式考虑己方场面代价，避免只按对方受到的伤害排序。

5.2 普通准备上限与关键分支保留

设普通出牌、物品或支援者分支的原始分数为 v，上限函数为

$$f(v) = \begin{cases} v & v \geq 10^5 \ \text{或} \ v \in \mathcal{W} \\ \min(v, 560) & \text{其他情况} \end{cases}, \quad \mathcal{W} = \{720, 900, 940, 950, 960\} \tag{3}$$

白名单在当前实现中保留特定保护换位、救场检索、回收或其他关键分支的原始高分。其设计意图是让普通准备动作和关键操作具有不同的优先级语义，而不是让所有非攻击动作都受同一上限约束。进化与贴能另有独立评分，不由式（3）统一截断。

5.3 当前实现中的节奏控制

这一评分体系用显式规则表达“何时继续准备、何时兑现攻击，以及哪些操作值得先处理”。它保持单次决策的可解释性，也使关键参数能够定位到具体函数。当前实现通过精确分值白名单保留关键分支，使相关优先级能够在函数入口与评分结果之间直接核对，由此形成以当前可见局面为中心的单步决策方法。

6 创新四：贯通进化供能与关键资源保护

6.1 以可攻击状态组织能量分配

能量评分同时考虑招式费用、现有能量以及追加一次供能后的可支付状态。`score_attach` 区分“当前已经能够攻击”“本次供能后可以攻击”和“仍需积累”的局面；当非 ex 反击分支中的 Bunnelby / Diggersby 满足条件时，提高将其转化为可攻击角色的优先级。

对前置进化角色，`charging_evolution` 检查其是否通向主攻击线路。当前形态已经可以使用低费招式，并不意味着额外能量没有用途：贴在前置形态上的能量可以随进化保留，因而仍然属于目标角色的准备进度。该设计把供能价值与进化依赖联系起来。

6.2 将特殊能量视为保护组件

Mist Energy 的评分同时考虑持有者、已有副本和当前威胁。主攻击角色尚无 Mist Energy 时，相关贴能分支给予较高优先级；在代码识别的手牌数量相关攻击条件下，已有一张时仍可提高第二张的配置价值；其他冗余配置则降低优先级。

这里体现了特殊能量的双重角色：既参与攻击费用，又参与保护条件。Hero’s Cape 也按场上角色与反击需求配置，而不是仅依据卡牌稀有度决定目标。图 4A 概括了从检索到回收的资源生命周期。

6.3 检索价值与弃置代价采用不同方向

检索与弃牌具有不同目标。检索处理器选择高价值候选；在卡牌弃置等对应上下文中，`discard_value` 表示丢失该牌的代价，按低分优先。当前表中 Mist Energy 为 99，Hero’s Cape 为 98，使其在该路径下倾向被保留。这一语义划分避免把“值得获得”和“适合丢弃”混成同一种排序含义。

资源回收同样依赖场面。Sacred Ash、Energy Recycler 等提供组件再利用；当可见牌库数量进入预设保护区间时，代码下调部分抽牌或检索分支，并保留特定回收的高优先级。资源价值因此包含当前攻击需求、后续展开与保护组件三个方面，而非只按手牌张数判断。

7 创新五：类型化决策与输出约束协同

7.1 把策略偏好落实为平台索引

动作选择首先形成策略偏好，最终却必须转换为平台候选的索引。设候选数量为 n_t，索引序列为 I_t，允许数量下界和上界分别为 l_t、u_t，则输出契约为

$$\mathrm{set}(I_t) \subseteq \{0, \cdots, n_t - 1\}, \quad |I_t| = |\mathrm{unique}(I_t)|, \quad l_t \le |I_t| \le u_t \tag{4}$$



图 4　资源生命周期与输出规范化。A：从组件检索到回收的设计联系；B：重复与越界索引的规范化构造示例，候选为 0-4，数量边界为 2-3。

常规分派路径通过 `finalize` 过滤越界项、去重、不足时补齐、超出时截断。图 4B 给出一个构造示例：候选索引为 0-4，需选 2-3 项，原始偏好含重复和越界值，经规范化后得到满足数量要求的索引。该示例用于解释算法，不来自对局记录。

7.2 分离选择语义与异常保护

不同类型的选择拥有独立处理器，但共享观察解析、候选对象定位和输出约束。入口与选择分派保留异常保护，在局部策略解析失败时进入兜底路径。当前结构将策略解释与平台交互集中在少数明确入口，便于追踪卡牌上下文和定位边界行为。

算法 **1** 局面驱动的选择与规范化流程。

```
输入：当前观察、合法候选、选择类型、数量边界
解析己方手牌与双方公开信息
构造防御、供能、攻击与资源上下文
按选择类型分派处理器
    主动作：结合攻击梯度与关键分支评分
    卡牌选择：按检索价值或弃置代价排序
    能量与其他选择：按对应上下文确定目标
常规分派异常时采用兜底偏好
过滤越界索引并去重
按最小数量补齐，再按最大数量截断
返回索引列表
```

7.3 内嵌元数据与轻量运行

卡牌与招式元数据随智能体封装，运行时直接读取包内信息；观察解析与规则评分只依赖 Python 标准库，不需要外部模型调用。卡组角色、攻击费用、招式文本和卡牌对象在同一实现中连接，便于从规则分支追踪到输入特征。内嵌元数据使规则解析与动作选择能够在统一的本地执行路径中完成。

8 设计创新与实现对应

表 2 将五项设计贡献与最终代码中的实现位置对应。该对应关系强调设计如何落地：每项创新都有可定位的输入、处理机制和输出作用。

表 **2** 设计贡献与代码模块对应。

创新点	核心设计	对应模块
机制互补双墙	分离伤害与效果保护，协调角色检索和换位	`effects_wall`、`need_effect_wall`、`score_retreat`
公开信号响应	由进化线路、公开手牌计数与 ex 构成调整操作关注点	`opp_hand_scale`、`non_ex_race`、`pick_value`
分层动作评分	普通准备上限、攻击梯度、关键分支与误伤成本共同排序	`score_main_opt`、`clamp_setup`、`score_play`
资源协同保护	可支付状态、进化供能、保护组件与回收共同决定资源用途	`score_attach`、`charging_evolution`、`discard_value`
类型化执行约束	上下文分派、对象定位与索引规范化共同完成接口动作	`choose`、`resolve_cid`、`finalize`

这些设计形成从卡组构筑到执行接口的连续联系：防御语义决定角色分工，公开信号决定当前需求，评分机制决定操作顺序，资源规则维持可执行条件，输出约束把偏好转为平台动作。项目的创新性体现在上述环节围绕同一局面的协同组织。

9 结论

本文围绕机制互补与局面响应设计 PTCG 规则智能体，将双墙角色、后备反击、分层动作评分、进化供能和关键资源保护连接为完整决策体系。其设计特点是将卡牌机制转化为具体角色职责，将可见信号转化为操作优先级，再通过类型化分派与输出规范化落实为可执行动作。

本项目的贡献集中于策略机制的组织与实现：双墙并行覆盖不同保护语义，公开信号支撑提前准备，分层评分表达当前动作关系，资源规则服务于目标角色，执行约束维持接口一致性。本文所述方法均以现有智能体实现为依据。

附录　卡组结构

卡组共 60 张，包括 14 张宝可梦、18 张能量、28 张训练家；训练家由物品 13 张、道具 1 张与支援者 14 张组成。卡牌承担角色防御、后备攻击、供能、检索、换位与回收等职责。

卡牌 **ID**	名称	数量
1	Basic {G} Energy	9
11	Mist Energy	3
16	Prism Energy	2
18	Grow Grass Energy	4
344	Dwebble	4
345	Crustle	4
414	Team Rocket's Articuno	2
1073	Bunnelby	2
1074	Diggersby	2
1086	Buddy-Buddy Poffin	4
1123	Switch	2
1129	Sacred Ash	1
1139	Energy Recycler	2
1152	Poké Pad	4
1159	Hero’s Cape	1
1182	Boss’s Orders	2
1210	Brock’s Scouting	2
1225	Hilda	2
1227	Lillie's Determination	2
1231	Dawn	3
1235	Waitress	3

资料来源与实现说明

[1] PTCG Arena. 卡牌数据. 机制名称、保护范围与卡牌编号的核对依据。

[2] Swcmb. ptcg-arena-cli. 平台工作流与候选索引契约的参考工具。

[3] 本项目智能体实现，归档 `main.py` 与 `deck.csv`，版本标识 wall37_ladder1。正文中的方法、参数与模块对应关系依据该实现整理。

设计、文字与排版整理使用 AI 辅助，智能体执行包内规则代码。